

Clinic Application Functionality and AI Integration Audit

What works, what is partial, what is absent, and where Clinical AI is actually deployed

Audit item	Value
Application	Clinic Management System (Flutter web + Flask/MySQL API)
Production URL	https://clinic.nalexustechologies.com
Audit date	16 June 2026 (Asia/Karachi)
Scope	Local repository, deployed server, live public endpoint, database state, and static code review
Security note	Passwords, API keys, database credentials, and secret values are intentionally excluded.

Executive Summary

The application has a broad and coherent clinic-management foundation: role-based authentication, clinic onboarding and approval, staff management, patient registration, appointments and token queues, doctor consultations, prescriptions, assistant workflows, pharmacy inventory and sales, payments, patient read-only portals, and multiple reports. The production API and database are healthy.

Production is not fully ready for normal use yet. The live database contains one clinic, two doctors, seven users, and subscription-plan seed data, but no patients, appointments, payments, prescriptions, vitals, reports, pharmacy stock, or sales. Therefore most workflows are implemented but have not been proven end-to-end with production data.

Clinical AI is deployed to the live doctor workflow. The production backend has the Clinical AI routes, an OpenAI API key, and model **gpt-5.2**. The verified release Flutter bundle containing the Clinical AI consultation interface is now served by Nginx.

Audit Evidence

Check	Observed result	Conclusion
Public health endpoint	HTTP 200; API=ok; database=ok	Production API and DB connection are live
Public web root	HTTP 200 text/html	Flutter web app is live
Backend service	clinic-backend.service active; Gunicorn on 127.0.0.1:5110	Backend process is running
Core services	Nginx and MySQL active	Required production services are running
Production AI config	provider=openai; model=gpt-5.2; API key present	OpenAI provider is configured
Production AI routes	4 /api/clinical-ai routes registered	AI backend is deployed
Live web bundle	Clinical AI interface present in /home/nalexus/clinic/frontend/main.dart.js	AI UI is deployed on the active website
Production security	MySQL and Gunicorn bound to localhost; UFW rules removed for 3306/5110	Database and backend are not publicly exposed
Production CORS	Production domain allowed; foreign test origin rejected	Browser API access is origin-restricted
Local Flutter analysis	No errors/warnings; 20 style-level info notices	Static analysis has no blocking issues
Local Flutter tests	1 boot test passed	Only minimal automated UI coverage exists
Local backend compile/tests	Python compile passed; 128 routes; 6 regression tests passed	Backend imports, routes, and corrected contracts succeed
Production warnings	No warning-level service logs in previous 7 days	No recent service warning evidence

Production Data State

Entity	Rows	Entity	Rows
Clinics	1	Users	7
Departments	2	Doctors	2
Subscription plans	3	Clinic subscriptions	0
Patients	0	Appointments	0
Payments	0	Prescriptions	0
Prescription medicines	0	Prescription lab tests	0
Patient vitals	0	Patient reports	0
Consultation drafts	0	Assistants	0
Pharmacy items	0	Pharmacy sales/items	0 / 0
Audit logs	0		

Interpretation: the system is deployed, but the absence of operational records means claims of complete production functionality should remain provisional until a controlled end-to-end acceptance test is run.

Functional Module Assessment

Module	Status	What is functional	Limitations / evidence gap
Authentication	Mostly functional	Login, /me, refresh, logout, forced first-login password change, profile update, role/clinic checks	Logout does not revoke JWT server-side
Clinic registration	Functional	Multi-step clinic, doctor, receptionist, pharmacy registration; temp accounts; pending approval	No production end-to-end test during audit; passwords shown once require operational handling
Super admin	Mostly functional	Dashboard, clinics, approvals, suspension, stats, revenue, subscription plan CRUD/assignment	No production subscription assignment acceptance test yet
Clinic admin	Mostly functional	Dashboard, staff, patients, appointments, revenue, reports, clinic settings, profile	Production end-to-end acceptance data is still absent
Doctor	Mostly functional	Dashboard, queue, consultation, patient profile, prescriptions, assistants, earnings, schedule, Clinical AI	Schedule appears informational; production has no consultations to verify
Assistant	Functional in code	Queue, call next, vitals, report URL upload, symptom draft, history, prescription print data	No production assistant records; report upload is URL-based rather than actual file storage
Receptionist	Mostly functional	Dashboard, patient CRUD, appointments, queue, booking, billing, receipts, reports	Production payment acceptance data is still absent
Pharmacy	Mostly functional	Inventory CRUD, low stock, expiry alerts, prescription orders, sales, invoices, reports	No production inventory/sales test data; no procurement/purchase-order workflow
Patient portal	Read-only partial	Dashboard, doctors, appointments, prescriptions, records, bills	No patient self-registration, appointment booking, cancellation, payment, profile update, or messaging workflow
Reports	Functional	Filtered reports, proportional charts, and PDF/CSV/JSON downloads	XLSX export is not included
Audit logging	Functional baseline	Successful authenticated mutations and login are logged; scoped admin read API exists	Before/after entity diffs and retention policy remain to be added
Subscriptions	Functional	Plan CRUD, assignment API/UI, subscription records, and doctor-limit linkage	Renewal/payment automation is not included
Clinic settings/profile	Functional	Clinic settings form, validation, save API, profile screen, and update API	Expanded branding/tax/holiday settings remain future scope

Clinical AI Audit

What Is Added

AI capability	Backend	Local frontend	Live frontend
Patient-history summary	Implemented: POST /api/clinical-ai/patient-summary	Used in doctor consultation screen	Deployed
Consultation draft	Implemented: POST /api/clinical-ai/consultation-assist	Used; draft can be reviewed/applied	Deployed
Medical entity extraction	Implemented: POST /api/clinical-ai/extract-medical-text	Service method exists but no user-facing screen calls it	API deployed; no visible workflow
Patient-friendly explanation	Returned by AI response schema	Displayed in consultation AI panel	Deployed
Red flags and questions	Returned by AI response schema	Displayed for doctor review	Deployed
Provider status	Implemented: GET /api/clinical-ai/status	Service method exists but not visibly used	Backend reachable only with doctor auth
Rule-based fallback	Implemented for missing key/provider errors	Consumed through same API	Backend available
Disease risk demos	Implemented: GET /risk-models and POST /risk-predict/{model}	Doctor Disease Risk Demo screen	Deployed

scikit-learn Disease Prediction Models

Demo model	Algorithm	Demo accuracy	Purpose
Diabetes risk	LogisticRegression	88.9%	Educational risk estimate from age, BMI, glucose, BP, and family history
Heart disease risk	RandomForestClassifier	85.5%	Educational risk estimate from cardiovascular risk-factor inputs
Stroke risk	DecisionTreeClassifier	79.1%	Educational risk estimate from age, comorbidity, glucose, BMI, and smoking inputs
Breast cancer screening risk	RandomForestClassifier	93.0%	Educational classifier using selected breast-cancer measurement features

Presentation line: We used scikit-learn to train disease prediction demo models on medical-style dataset features. The models predict an educational risk level based on patient inputs and report their held-out demonstration accuracy.

Model and Provider Placement

- **Provider:** OpenAI Responses API, called only from the Flask backend.
- **Configured model:** gpt-5.2 in local defaults, documentation, and production environment.
- **Backend implementation:** clinic_backend/app/services/clinical_ai_service.py.
- **scikit-learn implementation:** clinic_backend/app/services/disease_risk_service.py.
- **Backend routes:** clinic_backend/app/routes/clinical_ai_routes.py, registered in app/__init__.py.
- **Frontend implementation:** clinic_app/lib/core/services/clinical_ai_service.dart and the doctor consultation screen.
- **Production backend:** deployed under /home/nalexus/clinic/backend and configured with an OpenAI API key.
- **Production active frontend:** /home/nalexus/clinic/frontend; verified release bundle contains the AI UI.
- **Unused AI API:** extract-medical-text exists in the frontend service but has no visible screen/workflow.

AI Areas Not Added

- No AI symptom checker or pre-appointment triage for patients.
- No speech-to-text, ambient consultation recording, or medical transcription.
- No document OCR or automatic interpretation of uploaded lab-report files.
- No radiology/imaging AI, DICOM handling, or image model.
- No autonomous diagnosis, treatment, prescribing, appointment prioritization, or emergency decision engine.

- Disease risk models are educational screening demos only; they are not validated diagnostic models.
- No AI audit table, prompt/version tracking, output approval record, cost tracking, or per-clinic AI controls.
- No provider privacy/consent workflow, data-retention control, or verified BAA/compliance configuration in app code.

AI Safety Assessment

The code correctly treats AI output as an editable draft and includes a clinician-review safety notice. It does not automatically save AI output as a final record. However, production readiness still requires formal privacy approval, access logging, prompt/output auditability, model/version governance, failure monitoring, and clinical validation. A configured API key alone does not prove successful live model calls; the audit could not execute a real patient-summary request because there is no production doctor test credential and no patient/appointment data.

Confirmed Defects and Gaps

Previous severity	Finding	Resolution status	Verification
Critical	Active production web bundle omitted Clinical AI UI	Resolved	Verified release bundle deployed to active Nginx root and contains Clinical AI UI
High	Standalone billing contract mismatch	Resolved	Canonical/legacy aliases supported; appointment partial/paid state synchronized; regression tested
High	Forced password-change workflow/payload mismatch	Resolved	Change-password screen, redirects, correct old_password payload, and API test
High	Profile route missing	Resolved	Profile screen, route, update API, validation, and API test
Medium	Subscription/settings placeholders	Resolved	Plan CRUD, clinic assignment, and clinic settings UI/API implemented
Medium	Audit table unused	Resolved baseline	Central mutation/login writes and clinic-scoped admin read endpoint implemented/tested
Medium	Report charts/export placeholders	Resolved	Proportional charts and PDF/CSV/JSON downloads implemented
Medium	Payment detail API missing	Resolved	Authorized GET /api/payments/{id} deployed
Medium	Patient phone UI/backend mismatch	Resolved	Phone is visibly required and validated
Medium	MySQL publicly exposed	Resolved	MySQL binds 127.0.0.1; UFW 3306/5110 rules removed
Low	CORS allowed all origins	Resolved	Production origin accepted and foreign test origin rejected
Low	Automated tests minimal	Improved, ongoing	6 backend regression tests and Flutter boot test pass; broader role integration tests still recommended

Recommended Acceptance Test Sequence

- Create a dedicated non-production test clinic and accounts for every role.
- Approve the clinic, force password changes, and verify role navigation/authorization.
- Register a patient, book a paid and partial appointment, verify token and receipt.
- Run assistant queue, vitals, report upload, symptom draft, and call-next flows.
- Run doctor consultation, Clinical AI summary/draft, prescription, lab test, follow-up, and completion.
- Stock pharmacy inventory, dispense prescription, run walk-in sale, and verify stock/payment/invoice.
- Verify patient portal visibility and cross-clinic access denial.
- Verify every report against database totals and export outputs.
- Verify audit logs, backups, restoration, monitoring, rate limits, and secret rotation.

Overall Readiness

Area	Rating	Reason
Architecture	Good foundation	Clear Flask services/routes/models and role-based Flutter screens
Core workflow implementation	Mostly implemented	Broad code coverage, but limited production records and end-to-end proof
Clinical AI	Deployed for controlled use	OpenAI consultation support and scikit-learn disease risk demos deployed
Security and compliance	Improved baseline	Audit writes/read API, restricted CORS, local-only database, and forced password change implemented
Production readiness	Controlled acceptance testing	All identified application defects were corrected; operational/clinical acceptance and governance remain required

Code and Server Evidence Locations

- Local backend: clinic_backend/app, clinic_backend/migrations/schema.sql, clinic_backend/README.md.
- Local frontend: clinic_app/lib, especially routes, core/services, and screens.
- Production backend: /home/nalexus/clinic/backend.
- Production active frontend: /home/nalexus/clinic/frontend.
- Production backend service: /etc/systemd/system/clinic-backend.service.